Newton School of Technology

19

Agentic AI – Advanced Components

Moving beyond simple chatbots to intelligent agents

Newton School of Technology

PART 2

Agentic AI - Advanced Components

1.1 Today's Learning Journey

Newton School of Technology

01 **The Event Planner Analogy**
Understanding how AI agents differ from simple chatbots

02 **Memory Management**
Short-term vs. Long-term memory in AI systems

03 **Planning & Goal Decomposition**
Breaking complex tasks into manageable steps

1.1 Today's Learning Journey

Newton School of Technology

04 **The ReAct Framework**
Think → Act → Observe cycle

05 **State Management**
Maintaining context across interactions

1.2 The Event Planner Analogy

Newton School of Technology

Simple Chatbot

Answers once and leaves. Like asking a stranger for directions—they tell you once, then walk away. No follow-up, no memory.

- No context retention

AI Agent

Acts like an Event Planner. They don't just answer—they stick with you, handle problems, adjust plans, and see the entire event through from start to finish.

- End-to-end task completion

1.2 The Event Planner Analogy

Newton School of Technology

Four Key Traits of an AI Agent

It Remembers
Memory: Maintains context throughout the entire conversation and task execution

Breaks Down Goals
Planning: Decomposes complex objectives into manageable, sequential steps

Reacts to Problems
ReAct Loop: Adapts and adjusts when things don't go as expected

Follows Strict Process
LangGraph: Executes tasks through defined, structured workflows

1.3 Memory Management

Newton School of Technology

Short-term Memory
The "Sticky Note"

What It Does

Stores immediate information needed for the **current task only**. Temporary and disposable.

Real Example - When booking a flight, the agent remembers: "User wants flight AA123 on March 15th"

- Lifespan: Duration of current session/task

1.3 Memory Management

Newton School of Technology

Long-term Memory
The "Filing Cabinet"

What It Does

Stores permanent information that persists across sessions. Uses Vector Databases for efficient retrieval.

Real Example - The agent remembers forever: "User prefers aisle seats and vegetarian meals"

∞ Lifespan: Permanent (until explicitly deleted)

Key Insight: Short-term memory handles the "what" of current tasks, while long-term memory stores the "who" and "preferences" for personalized experiences.

1.4 Planning & Goal Decomposition

Newton School of Technology

The Analogy

"Building a Skyscraper"

One floor at a time, but with a Site Manager.

Why It Matters

- Makes impossible tasks possible
- Enables step-by-step progress tracking
- Allows error recovery at each step
- Creates clear execution roadmap

1.4 Planning & Goal Decomposition

Newton School of Technology

Example: "Plan My Vacation"

1. **Find Flights**
   Search for available flights matching dates and budget

↓

2. **Book Hotel**
   Reserve accommodation near desired locations

3. **List Restaurants**
   Find dining options matching cuisine preferences

Quick Exercise: Break down "Bake a Cake" → 1) Buy Ingredients → 2) Mix Batter → 3) Put in Oven

1.5 The ReAct Framework

Newton School of Technology

The Cycle: THINK → ACT → OBSERVE

THINK

Analyze the situation and decide what action to take

→

ACT

Execute the chosen action (search, calculate, call API)

→

OBSERVE

Review the results and determine next steps

Real Example

1. **THINK:** "I need to search for 'best Italian restaurants'"
2. **ACT:** Search executed
3. **OBSERVE:** "No results found"
4. **THINK AGAIN:** "Try 'top rated pasta places' instead"

The cycle repeats until the task is complete. Each observation feeds back into the thinking phase, allowing the agent to adapt and refine its approach.

1.6 State Management (The Order Pad)

Newton School of Technology

What is State?

State is a Memory Object that stores all the context and information the agent needs to remember throughout the interaction.

Think of it like a waiter's order pad—constantly updated as the conversation progresses.

Why State Matters

- Prevents the agent from forgetting previous context
- Enables multi-turn conversations
- Tracks progress through complex workflows
- Validates inputs and maintains data integrity

1.6 State Management (The Order Pad)

Newton School of Technology

Example: Pizza Delivery Bot

U "I want to order a pizza with shoe topping"

A **State Check:** "Shoe" is not in valid toppings list
→ **REJECT & LOOP BACK**

A "Sorry, 'shoe' is not a valid topping. Please choose from: Cheese, Pepperoni, Mushrooms..."

U "Cheese, please"

A **State Check: "Cheese" ✓ Valid topping**
→ **ACCEPT & PROCEED**

A "Great! Added cheese topping. What size would you like?"

State validation ensures data quality and guides users toward successful task completion.

Newton School of Technology

2. Building Agents with LangGraph

- Python Library
- Graph-Based
- Agentic AI

2.1 What is LangGraph?

The Library for Agentic Workflows

LangGraph is a Python library designed to build stateful, multi-actor applications with LLMs.

Moving from a **simple chat** to a **structured graph of logic** . Instead of linear conversations, you build decision trees where agents can route themselves based on conditions.

Cyclical Flows

Conditional Logic

langgraph_intro.py

```
# Traditional Chat: Linear
response = llm.invoke("Book a flight")

# LangGraph: Cyclical & Stateful
workflow = StateGraph(State)
workflow.add_node("find_flight", find_flight)
workflow.add_node("check_price", check_price)
workflow.add_node("book", book_flight)
workflow.add_conditional_edges(
"check_price",
route_based_on_price,
{"book": "book", "ask": "ask_user"}
)
app = workflow.compile()
```

2.2 The 'State'

Newton School of Technology

The Shared Diary Passed Between Steps

Every agent needs a **shared memory object**—the State—that persists and evolves as the workflow executes.
Think of it as a diary that each "worker" (node) reads from and writes to, passing context from one step to the next.

Why State Matters

- **Persistence:** Data survives across function calls
- **Communication:** Nodes share information seamlessly
- **Type Safety:** TypedDict ensures structured data

Best Practice: Define all possible fields your workflow might need upfront.

2.2 The 'State'

The Shared Diary Passed Between Steps

state_definition.py

```python
from typing import TypedDict, Annotated
from operator import add

# Define the State structure
class TravelState(TypedDict):
    flight_price: int  # Current price
    decision: str  # 'book' or 'ask'
    destination: str  # Where to go
    user_approved: bool  # Permission flag
    messages: Annotated[list, add]   # Chat history

# Initialize state
state = TravelState(
flight_price=0,
decision="pending",
destination="",
user_approved=False,
messages=[]
)
```

Key Fields Explained

flight_price: Stores cost

destination: Target location

decision: Routing flag

user_approved: Permission

2.3 'Nodes' (The Workers)

Newton School of Technology

Functions That Perform Work and Update State

Nodes are Python functions that receive the current state, perform operations, and return updates.

Each node is a **discrete unit of work**—like checking a price, calling an API, or asking the user a question.

Node Characteristics

- **Input:** Receives current state
- **Process:** Executes logic
- **Output:** Returns state updates

Pro Tip: Keep nodes focused on a single responsibility.

2.3 'Nodes' (The Workers)

Newton School of Technology

Functions That Perform Work and Update State

nodes_example.py

```python
# Node 1: Find flight and set price
def find_flight(state: TravelState):
    price = search_flights(state["destination"])
    return {"flight_price": price}

# Node 2: Check price and decide
def check_price(state: TravelState):
    price = state["flight_price"]
    if price > 500:
        return {"decision": "ask_user"}
    else:
        return {"decision": "book"}

# Node 3: Book the flight
def book_flight(state: TravelState):
    confirmation = book(state["flight_price"])
    return {"messages": [f"Booked! {confirmation}"]}

# Node 4: Ask user for permission
def ask_user(state: TravelState):
    msg = f"Price is ${state['flight_price']}. Approve?"
    return {"messages": [msg]}
```

2.4 'Edges' (The Logic Gates)

Newton School of Technology

Determining the Path Based on Current State

Edges connect nodes and define the flow of execution. They determine which node runs next. **Conditional Edges** are the magic—they route the agent to different functions based on state values.

Types of Edges

→ Standard Edges
Fixed paths: Node A → Node B

🔀 Conditional Edges
Dynamic paths based on state

🧠 **Think of it as:** A GPS that dynamically reroutes based on traffic conditions.

2.4 'Edges' (The Logic Gates)

Newton School of Technology

Determining the Path Based on Current State

conditional_edges.py

```
# Routing function for conditional edge

def route_based_on_price(state: TravelState):
    decision = state["decision"]
    if decision == "book":
        return     "book"                    # Route to book_flight
    else:
        return     "ask"                     # Route to ask_user

# Add conditional edge to workflow
workflow.add_conditional_edges(
source="check_price", # From this node
path=route_based_on_price, # Routing function

"book": "book_flight", # If returns "book"
"ask": "ask_user"# If returns "ask"
}
)
```

How It Works

check_price → route_based_on_price → book_flight

2.5 The Workflow Graph

Newton School of Technology

Part 1: Connecting Everything with StateGraph

The **StateGraph i**s the builder that assembles your nodes and edges into a cohesive workflow. Think of it as the canvas where you paint your agent's decision tree—connecting workers (nodes) with pathways (edges).

The Builder Pattern

- **Create** a StateGraph with your State type
- **Add** nodes (worker functions)
- **Connect** nodes with edges
- **Compile** into an executable app

Analogy: Like building with LEGO blocks—each piece connects to form something greater

2.5 The Workflow Graph

Part 1: Connecting Everything with StateGraph

workflow_builder.py

```
from langgraph.graph import StateGraph, END

# Step 1: Create the StateGraph
workflow = StateGraph(TravelState)

# Step 2: Add nodes (register workers)
workflow.add_node("find_flight", find_flight)
workflow.add_node("check_price", check_price)
workflow.add_node("book_flight", book_flight)
workflow.add_node("ask_user", ask_user)

# Step 3: Add edges (connect nodes)
workflow.add_edge("find_flight", "check_price")
workflow.add_conditional_edges(
"check_price",
route_based_on_price,
{"book": "book_flight", "ask": "ask_user"}
)
workflow.add_edge("book_flight", END)
workflow.add_edge("ask_user", END)
```

2.5 The Workflow Graph

Newton School of Technology

Part 2: Three Essential Commands

add_node

Registers a worker function as a node in the graph.

```
# Syntax
graph.add_node(
"node_name",
function_name
)

# Example
workflow.add_node(
"check_price",
check_price
)
```

What it does: Adds a processing step that can read/write state.

add_conditional_edges

Creates dynamic routing based on state values.

```
# Syntax
graph.add_conditional_edges(
"source_node",
routing_function,
{"key": "target"}
)

# Example
workflow.add_conditional_edges(
"check_price",
route_func,
{"book": "book"}
)
```

What it does: Enables decision-making in the workflow.

set_entry_point

Defines where the workflow execution begins.

```
# Syntax
graph.set_entry_point(
    "node_name"
)

# Example
workflow.set_entry_point(
    "find_flight"
)

# Alternative
workflow.set_conditional_entry_point(
routing_function
)
```

What it does: Specifies the starting node.

2.6 Case Study: Travel Agent Bot

Newton School of Technology

Scenario & Implementation

The Scenario

Build an agent that:

- **Finds** available flights
- **Checks** if price > $500
- **Asks** for permission if expensive
- **Books** automatically if cheap

Decision Logic

```
if flight_price > 500:
    decision = "ask_user"
    → Route to ask_user node
else:
    decision = "book"
    → Route to book_flight node
```

💡 **Real-world use case:** Automating expense policies, approval workflows, or any conditional process.

2.6 Case Study: Travel Agent Bot

Newton School of Technology

Scenario & Implementation

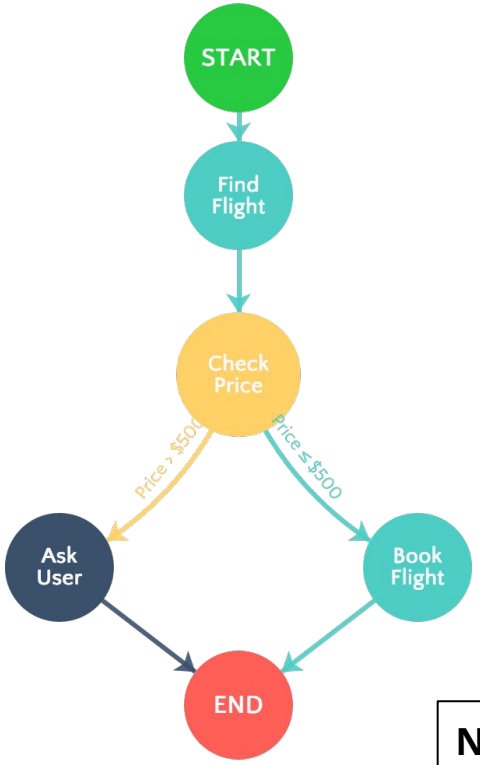
travel_agent_complete.py

```python
# Complete implementation

from langgraph.graph import StateGraph, END
from typing import TypedDict

# 1. Define State
classTravelState(TypedDict):
    flight_price: int
    decision: str
    messages: list

# 2. Define Nodes
def find_flight(state): ...
def check_price(state): ...
def book_flight(state): ...
def ask_user(state): ...

# 3. Build & Compile
workflow = StateGraph(TravelState)
workflow.add_node("find", find_flight)
workflow.add_node("check", check_price)
workflow.add_node("book", book_flight)
workflow.add_node("ask", ask_user)
workflow.add_edge("find", "check")
workflow.add_conditional_edges("check", route_func, {...})
workflow.set_entry_point("find")

app = workflow.compile()
```

2.6 Case Study: Travel Agent Bot

Newton School of Technology

Visual Flow & Decision Tree



Flow Explanation

1. **Start:** Agent begins at "Find Flight"
2. **Search:** Finds available flights
3. **Decision:** Price check at conditional node
4. **Route:** Splits based on price
5. **End:** Either books or asks

Key Insights

- ✓ **Cyclical:** Can loop back for user input
- ✓ **Stateful:** Price persists across nodes
- ✓ **Dynamic:** Routes change based on data

Next Steps: Add more nodes for hotel booking, car rentals, etc.

Newton School of Technology

Start Building Your Own Agents

LangGraph empowers you to create intelligent, stateful agents.
Apply these concepts to build workflows that think, decide, and act.

Think

Stateful reasoning across steps

Decide

Conditional routing logic

Act

Execute and update state

Thank You !!

See You Guys in Next Session :)